

Ex. No.: 5

Date:

A PYTHON PROGRAM TO IMPLEMENT MULTI LAYER PERCEPTRON WITH BACK PROPOGATION

Aim:

To implement multilayer perceptron with back propagation using python.

Algorithm:

Step 1: Import the Necessary Libraries

- Import pandas as pd.
- Import numpy as np.

Step 2: Read and Display the Dataset

- Use ``pd.read_csv("banknotes.csv")`` to read the dataset.
- Assign the result to a variable (e.g., ``data``).
- Display the first ten rows using ``data.head(10)``.

Step 3: Display Dataset Dimensions

- Use the ``.shape`` attribute on the dataset (e.g., ``data.shape``).

Step 4: Display Descriptive Statistics

- Use the ``.describe()`` function on the dataset (e.g., ``data.describe()``).

Step 5: Import Train-Test Split Module

- Import ``train_test_split`` from ``sklearn.model_selection``.

Step 6: Split Dataset with 80-20 Ratio

- Assign the features to a variable (e.g., ``X = data.drop(columns='target')``).
- Assign the target variable to another variable (e.g., ``y = data['target']``).
- Use ``train_test_split`` to split the dataset into training and testing sets with a ratio of 0.2.

- Assign the results to ``x_train``, ``x_test``, ``y_train``, and ``y_test``.

Step 7: Import MLPClassifier Module

- Import ``MLPClassifier`` from ``sklearn.neural_network``.

Step 8: Initialize MLPClassifier

- Create an instance of ``MLPClassifier`` with ``max_iter=500`` and ``activation='relu'``.
- Assign the instance to a variable (e.g., ``clf``).

Step 9: Fit the Classifier

- Fit the model using ``clf.fit(x_train, y_train)``.

Step 10: Make Predictions

- Use the ``.predict()`` function on ``x_test`` (e.g., ``pred = clf.predict(x_test)``).
- Display the predictions.

Step 11: Import Metrics Modules

- Import ``confusion_matrix`` from ``sklearn.metrics``.
- Import ``classification_report`` from ``sklearn.metrics``.

Step 12: Display Confusion Matrix

- Use ``confusion_matrix(y_test, pred)`` to generate the confusion matrix.
- Display the confusion matrix.

Step 13: Display Classification Report

- Use ``classification_report(y_test, pred)`` to generate the classification report.
- Display the classification report.

Step 14: Repeat Steps 9-13 with Different Activation Functions

- Initialize ``MLPClassifier`` with ``activation='logistic'``.

- Fit the model and make predictions.
- Display the confusion matrix and classification report.
- Repeat for `activation='tanh'`.
- Repeat for `activation='identity'`.

Step 15: Repeat Steps 7-14 with 70-30 Ratio

- Use `train\_test\_split` to split the dataset into training and testing sets with a ratio of 0.3.
- Assign the results to `x\_train`, `x\_test`, `y\_train`, and `y\_test`.
- Repeat Steps 7-14 with the new training and testing sets.

PROGRAM:

```
import pandas as pd
import numpy as np
bnotes = pd.read_csv('../input/banknotes-dataset/bank_note_data.csv')
bnotes.head(10)
```

bnotes.shape	:	Image.Var	Image.Skew	Image.Curt	Entropy	Class
(1372, 5)	0	3.62160	8.6661	-2.80730	-0.44699	0
	1	4.54590	8.1674	-2.45860	-1.46210	0
	2	3.86600	-2.6383	1.92420	0.10645	0
	3	3.45660	9.5228	-4.01120	-3.59440	0
	4	0.32924	-4.4552	4.57180	-0.98880	0
	5	4.36840	9.6718	-3.96060	-3.16250	0
	6	3.59120	3.0129	0.72888	0.56421	0
	7	2.09220	-6.8100	8.46360	-0.60216	0
	8	3.20320	5.7588	-0.75345	-0.61251	0
	9	1.53560	9.1772	-2.27180	-0.73535	0

```
bnotes.describe(include='all')
```

	Image.Var	Image.Skew	Image.Curt	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

```
x = bnotes.drop('Class',axis=1)
```

```
y = bnotes['Class']
```

```
print(x.head(2))
```

```
print(y.head(2))
```

```

      Image.Var  Image.Skew  Image.Curt  Entropy
0      3.6216      8.6661     -2.8073 -0.44699
1      4.5459      8.1674     -2.4586 -1.46210
0      0
1      0
Name: Class, dtype: int64

```

```
from sklearn.model_selection import train_test_split
```

```
#train_test ratio = 0.2
```

```
x_train,x_test,y_train,y_test =
```

```
train_test_split(x,y,test_size=0.2) from sklearn.neural_network
```

```
import MLPClassifier
```

```
# activation function : relu
```

```
mlp = MLPClassifier(max_iter=500,activation='relu')
```

```
mlp.fit(x_train,y_train)
```

```
MLPClassifier(max_iter=500)
```

```
pred = mlp.predict(x_test)
```

```
print(pred)
```

```
[0 0 0 1 0 0 0 1 0 0 1 1 1 0 1 0 0 0 0 1 0 1 0 0 1 0 1 0 1 0 1 1 1 1 0
0 1 0 0 0 1 0 1 1 1 1 0 1 0 0 1 1 0 1 0 1 1 1 0 0 1 1 1 1 0 1 0 1 1 0 0 0
0 1 1 0 1 1 1 0 1 0 0 0 1 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 1 0 1 0 1 0 0
0 0 0 0 1 0 1 1 1 1 1 0 0 0 0 1 0 0 0 1 0 0 0 1 0 1 1 0 0 0 1 0 1 0 1 0 1
1 1 1 0 1 0 1 0 0 1 1 0 1 1 0 1 1 0 0 0 0 0 0 0 1 1 1 1 1 0 1 1 1 1 0 1 0 1
0 1 0 1 0 0 0 1 0 1 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 1 0 1 0 0 0 0 0 1 1 1
1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 1 1 0
0 0 0 1 0 0 0 0 1 1 0 0 1 1 0 0]
```

```
from sklearn.metrics import classification_report, confusion_matrix
confusion_matrix(y_test, pred)
```

```
array([[153, 0], [0, 122]])
```

```
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	153
1	1.00	1.00	1.00	122
accuracy			1.00	275
macro avg	1.00	1.00	1.00	275
weighted avg	1.00	1.00	1.00	275

activation function : logistic

```
mlp = MLPClassifier(max_iter=500, activation='logistic')
```

```
mlp.fit(x_train, y_train)
```

MLPClassifier(activation='logistic', max_iter=500)

```
pred = mlp.predict(x_test)
```

```
print(pred)
```

```
[0 0 0 1 0 0 0 1 0 0 1 1 1 0 1 0 0 0 0 1 0 1 0 0 1 0 1 0 1 0 1 1 1 1 0
0 1 0 0 0 1 0 1 1 1 1 0 1 0 0 1 1 0 1 0 1 1 1 0 0 1 1 1 1 0 1 0 1 1 0 0 0
0 1 1 0 1 1 1 0 1 0 0 0 1 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 1 0 1 0 1 0 0
0 0 0 0 1 0 1 1 1 1 1 0 0 0 0 1 0 0 0 1 0 0 0 1 0 1 1 0 0 0 1 0 1 0 1 0 1
1 1 1 0 1 0 1 0 0 1 1 0 1 1 0 1 1 0 0 0 0 0 0 0 1 1 1 1 1 0 1 1 1 1 0 1 0 1
0 1 0 1 0 0 0 1 0 1 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 1 0 1 0 0 0 0 0 1 1 1
1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 1 1 0
0 0 0 1 0 0 0 0 1 1 0 0 1 1 0 0]
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
confusion_matrix(y_test, pred)
```

```
array([[153,  0],
       [ 0, 122]])
```

```
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	153
1	1.00	1.00	1.00	122
accuracy			1.00	275
macro avg	1.00	1.00	1.00	275
weighted avg	1.00	1.00	1.00	275

activation function : tanh

```
mlp = MLPClassifier(max_iter=500, activation='tanh')
```

```
mlp.fit(x_train, y_train)
```

```
pred = mlp.predict(x_test)
```

```
print(pred)
```

```
[0 0 0 1 0 0 0 1 0 0 1 1 1 0 1 0 0 0 0 1 0 1 0 0 1 0 1 0 1 0 1 1 1 1 0
 0 1 0 0 0 1 0 1 1 1 1 0 1 0 0 1 1 0 1 0 1 1 1 0 0 1 1 1 1 0 1 0 1 1 0 0 0
 0 1 1 0 1 1 1 0 1 0 0 0 1 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 1 0 1 0 1 0 0
 0 0 0 0 1 0 1 1 1 1 1 0 0 0 0 1 0 0 0 0 1 0 0 0 1 0 1 1 0 0 0 1 0 1 0 1 0 1
 1 1 1 0 1 0 1 0 0 1 1 0 1 1 0 1 1 0 0 0 0 0 0 0 1 1 1 1 1 0 1 1 1 0 1 0 1
 0 1 0 1 0 0 0 1 0 1 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 1 0 1 0 0 0 0 0 1 1 1
 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 1 1 0
 0 0 0 1 0 0 0 0 1 1 0 0 1 1 0 0]
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
confusion_matrix(y_test, pred)
```

```
array([[153,  0],
       [ 0, 122]])
```

```
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	153
1	1.00	1.00	1.00	122
accuracy			1.00	275
macro avg	1.00	1.00	1.00	275
weighted avg	1.00	1.00	1.00	275

activation function : identity

```
mlp = MLPClassifier(max_iter=500,activation='identity')
```

```
mlp.fit(x_train,y_train)
```

```
MLPClassifier(activation='identity', max_iter=500)
```

```
pred = mlp.predict(x_test)
```

```
print(pred)
```

```
[0 0 0 1 0 0 0 0 0 0 1 1 1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 1 1 0 1 1 1 1 0
 0 1 0 0 0 1 0 1 1 1 1 0 1 0 0 1 1 0 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 0 0 0 0
 0 1 1 0 1 1 1 0 1 0 0 0 1 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 1 0 1 0 1 0 0
 0 0 0 0 1 0 1 1 1 1 1 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 0 1 0 1
 1 1 1 0 1 0 1 0 0 1 1 0 1 1 0 1 1 0 0 0 0 0 0 0 1 1 1 1 1 0 1 1 1 1 0 1 0 1
 0 1 0 1 0 0 0 1 0 1 1 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 1 0 1 0 0 0 0 0 1 1 1
 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 0 0 0 0 0 1 0 0 0 0 1 0 0 0 1 1 0
 0 0 0 1 0 0 0 0 1 1 0 0 1 1 0 0]
```

```
from sklearn.metrics import classification_report,confusion_matrix
```

```
confusion_matrix(y_test,pred)
```

```
array([[150,  3],
       [ 3, 119]])
```

```
print(classification_report(y_test,pred))
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	153
1	0.98	0.98	0.98	122

accuracy			0.98	275
macro avg	0.98	0.98	0.98	275
weighted avg	0.98	0.98	0.98	275

#train_test ratio = 0.3

x_train,x_test,y_train,y_test =

train_test_split(x,y,test_size=0.3) from sklearn.neural_network

import MLPClassifier

activation function : relu

mlp =

MLPClassifier(max_iter=500,activation='relu')

mlp.fit(x_train,y_train)

MLPClassifier(max_iter=500)

pred = mlp.predict(x_test)

print(pred)

```
[0 1 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1
 1 1 0 1 1 1 1 0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 0 1 0 1 1 0 1 1 1 0 0 1 0
 1 1 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0 1 0 0 0 1 0 0 1 1 1 1
 1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 1 1 1 0 1 0 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1 1
 0 1 0 1 0 1 0 1 1 0 0 1 1 0 0 1 1 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 1 0 0 0 0
 0 0 0 1 0 1 1 0 0 1 1 1 0 0 0 1 0 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0
 1 0 0 0 1 1 1 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 0 0 0 1
 0 0 0 1 0 0 0 0 1 0 1 1 0 0 0 1 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0 0 1
 0 1 1 0 1 0 1 1 0 1 0 0 0 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 0 1 0 0 1
 0 1 0 1 1 1 0 1 0 0 1 0 1 1 0 0 0 1 0 1 1 1 1 0 0 0 1 0 1 0 0 0 0 0 1 0 0
 1 0 1 1 0 1 0 0 0 0 1 1 1 0 1 1 0 1 0 0 1 0 0 1 1 1 0 0 0 1 1 0 0 1 1 0 1
 0 1 1 0 0]
```

from sklearn.metrics import classification_report,confusion_matrix

confusion_matrix(y_test,pred)

```
array([[220,  0],
       [ 0, 192]])
```

print(classification_report(y_test,pred))

	precision	recall	f1-score	support
0	1.00	1.00	1.00	220
1	1.00	1.00	1.00	192
accuracy			1.00	412
macro avg	1.00	1.00	1.00	412
weighted avg	1.00	1.00	1.00	412

activation function : logistic

```
mlp = MLPClassifier(max_iter=500,activation='logistic')
```

```
mlp.fit(x_train,y_train)
```

```
MLPClassifier(max_iter=500,activation='logistic')
```

```
pred = mlp.predict(x_test)
```

```
print(pred)
```

```
MLPClassifier(max_iter=500,activation='tanh')
```

```
[0 1 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1  
1 1 0 1 1 1 1 0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 0 1 0 1 1 0 1 1 1 0 0 1 0  
1 1 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0 1 0 0 0 1 0 0 1 1 1 1  
1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 1 1 1 0 1 0 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1 1  
0 1 0 1 0 1 0 1 1 0 0 1 1 0 0 1 1 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 1 0 0 0 0  
0 0 0 1 0 1 1 0 0 1 1 1 0 1 0 1 0 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0  
1 0 0 0 1 1 1 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 0 1 0 0 0 1  
0 0 0 1 0 0 1 0 1 0 1 1 0 0 0 1 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0 0 1  
0 1 1 0 1 0 1 1 0 1 0 0 0 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 0 1 0 0 1  
0 1 0 1 1 1 0 1 0 0 1 0 1 1 0 0 0 1 0 1 1 1 1 0 0 0 1 0 1 0 0 0 0 1 0 0  
1 0 1 1 0 1 0 0 0 0 1 1 1 0 1 1 0 1 0 0 1 0 0 1 1 1 0 0 0 1 1 0 0 1 1 0 1  
0 1 1 0 0]
```

```
from sklearn.metrics import classification_report,confusion_matrix
```

```
confusion_matrix(y_test,pred)
```

activation function : tanh

```
mlp = MLPClassifier(max_iter=500,activation='tanh')
```

```
mlp.fit(x_train,y_train)
```

```
pred = mlp.predict(x_test)
```

```
print(pred)
```

```
[0 1 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1
1 1 0 1 1 1 1 0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 0 1 0 1 1 0 1 1 1 0 0 1 0
1 1 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0 1 0 0 0 1 0 0 1 1 1 1
1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 1 1 1 0 1 0 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1 1
0 1 0 1 0 1 0 1 1 0 0 1 1 0 0 1 1 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 1 0 0 0 0
0 0 0 1 0 1 1 0 0 1 1 1 0 0 0 1 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0
1 0 0 0 1 1 1 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 0 1 0 0 0 1
0 0 0 1 0 0 0 0 1 0 1 1 0 0 0 1 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0 0 1
0 1 1 0 1 0 1 1 0 1 0 0 0 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 0 1 0 0 1
0 1 0 1 1 1 0 1 0 0 1 0 1 1 0 0 0 1 0 1 1 1 1 0 0 0 1 0 1 0 0 0 0 0 1 0 0
1 0 1 1 0 1 0 0 0 0 1 1 1 0 1 1 0 1 0 0 1 0 0 1 1 1 0 0 0 1 1 0 0 1 1 0 1
0 1 1 0 0]
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
confusion_matrix(y_test, pred)
```

```
array([[220,  0],
       [ 0, 192]])
```

```
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	220
1	1.00	1.00	1.00	192
accuracy			1.00	412
macro avg	1.00	1.00	1.00	412
weighted avg	1.00	1.00	1.00	412

activation function : identity

```
mlp = MLPClassifier(max_iter=500, activation='identity')
```

```
mlp.fit(x_train, y_train)
```

```
MLPClassifier(max_iter=500, activation='identity')
```

```
pred = mlp.predict(x_test)
```

```
print(pred)
```

```
[0 1 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1
1 1 0 1 1 1 1 0 1 0 1 0 1 1 0 0 0 1 1 1 0 0 1 0 0 1 0 1 1 0 1 1 1 0 0 0 0
1 1 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0 1 0 0 0 1 0 0 1 1 1 1
1 1 1 0 1 1 0 1 1 1 1 1 1 1 0 1 1 1 1 0 1 0 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1
0 1 0 1 0 1 0 1 1 0 0 1 1 0 0 1 1 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 1 0 0 0 0
0 0 0 1 0 1 1 0 0 1 1 1 0 1 0 1 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0
1 0 0 0 1 1 1 1 0 0 0 0 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 0 0 0 1
0 0 0 1 0 0 1 0 1 0 1 1 0 0 0 1 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0 0 1
0 1 1 0 1 0 1 1 0 1 0 0 0 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 0 1 0 0 1
0 1 0 1 1 1 1 0 1 0 0 1 0 1 1 0 0 0 1 0 1 1 1 1 0 0 0 1 0 1 0 0 0 0 1 0 0
1 0 1 1 0 1 0 0 0 0 1 1 1 0 1 1 0 1 0 0 1 1 1 0 0 0 1 1 0 0 0 1 1 0 0 1
0 1 1 0 0]
```

```

from sklearn.metrics import classification_report, confusion_matrix
confusion_matrix(y_test, pred)
array([[217,  3],
       [ 1, 191]])

print(classification_report(y_test, pred))

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	220
1	0.98	0.99	0.99	192
accuracy			0.99	412
macro avg	0.99	0.99	0.99	412
weighted avg	0.99	0.99	0.99	412

RESULT:

Thus the python program to implement multi layer perceptron with back propagation on the given dataset(banknotes.csv) has been executed successfully and it's results have been analyzed successfully for different activation functions(relu,logistic,tanh,identity) with two different training-testing ratios(0.2 and 0.3)